COM32 · SESIÓN S04 · SEMANA 4 · CORTE 1 · RA1

SQL dentro de PL/SQL y cursores implícitos

El bloque deja de trabajar solo con variables: consulta, inserta, modifica y elimina filas de las tablas, y sabe cuántas filas afectó cada sentencia.

PROGRAMACIÓN EN BASE DE DATOS · JUEVES 1 DE OCTUBRE DE 2026 · 18:00 – 22:00 · SEDE PLAZA DE LA PAZ

Kevin Erney De Alba Pomárico · Institución Universitaria de Barranquilla
kerneydealba@unibarranquilla.edu.co

Proyectar mientras llega el grupo. Pedir que abran FreeSQL (freesql.com) y descarguen COM32_S04_ejemplos.sql del aula. Quien tenga el espacio APEX aprobado lo deja abierto en otra pestaña.

PLAN DE LA SESIÓN

Agenda de la sesión

HORA	BLOQUE	CONTENIDO
18:00	Apertura	Compromisos y tablas de la sesión
18:10	Bloque 1	SELECT INTO: una fila, funciones de grupo y sus errores
18:45	Bloque 2	INSERT, UPDATE y DELETE en el bloque; COMMIT y ROLLBACK
19:20	Descanso	10 minutos
19:30	Bloque 3	Cursor implícito: SQL%ROWCOUNT, SQL%FOUND y SQL%NOTFOUND
19:55	Proyecto integrador	Estructura de la entrega y Entrega 1; esquema en APEX
20:25	Taller 04	Calificable, individual, en FreeSQL
21:55	Cierre	Síntesis y entregas

2 min. Ruta mínima si el tiempo no alcanza: la diapositiva 23 (SQL%ROWCOUNT después del COMMIT) queda para lectura, los ejercicios de los bloques 1 y 2 se reducen al paso 1 y el bloque del proyecto se reduce a las diapositivas 27 y 28. El taller conserva sus 90 minutos.

Lo trabajado en la S03

Ya disponible

- Tipos escalares y su tamaño: `NUMBER(p,s)`, `VARCHAR2`, `DATE`
- Variables ancladas con `%TYPE`
- `NULL` en operaciones, `NVL` y `COALESCE`
- Conversiones explícitas con máscara
- Un primer `SELECT INTO` de una fila

Lo que agrega esta sesión

- Qué sentencias SQL acepta un bloque y cuáles no
- Cuándo falla un `SELECT INTO`
- Escribir en las tablas desde el bloque
- Confirmar o deshacer los cambios
- Saber cuántas filas afectó cada sentencia

Código de la sesión: `COM32_S04_ejemplos.sql`, en el aula. Cada diapositiva con ejemplo indica su bloque (B00 a B09). Se ejecuta un bloque a la vez, incluida la barra final.

Evaluación de la S03. El Taller 03 no se publicó. Los contenidos de la S03 se evalúan en el Taller 04 de hoy, que integra las dos sesiones (diapositiva 30).

3 min. La S03 terminó con %TYPE y SELECT INTO de una fila: esta sesión parte de ahí. Anunciar que el Taller 03 no se aplicó y que su contenido entra en el Taller 04.

APERTURA código: COM32_S04_ejemplos.sql · B00

La tabla productos de la S03 crece

```
ALTER TABLE productos MODIFY (existencias DEFAULT 0 NOT NULL);   -- ya existe
ALTER TABLE productos ADD CONSTRAINT ck_productos_existencias CHECK (existencias >= 0);
ALTER TABLE productos ADD (existencias_minima NUMBER(3) DEFAULT 5 NOT NULL);  -- nueva
ALTER TABLE productos ADD (estado_inventario VARCHAR2(8) GENERATED ALWAYS AS (
  CASE WHEN existencias = 0 THEN 'AGOTADO'
       WHEN existencias < existencias_minima THEN 'REPONER' ELSE 'OK' END) VIRTUAL);
```

CODIGO	NOMBRE	PRECIO	PRECIO_CON_IVA	EXISTENCIAS	MÍNIMA	ESTADO
1	Arroz 500 g	2800	2800	40	10	OK
2	Café 250 g	9500	9975	12	5	OK
4	Aceite 1 L	15900	18921	8	10	REPONER
5	Panela 500 g	3200	3200	25	10	OK
6	Leche 1 L	4600	4600	0	5	AGOTADO
7	Jabón de baño	5800	6902	15	5	OK
8	Atún en lata 170 g	7300	7665	30	10	OK

- MODIFY cambia una columna que ya existe; ADD agrega una nueva. `estado_inventario` es virtual, como `precio_con_iva`: Oracle la calcula con CASE.
- B00c y B00d cargan con INSERT los 7 productos y 8 clientes base.
- El código 9 está libre: el azúcar se inserta en B04.

4 min. Todos ejecutan B00a a B00e. Antes del MODIFY, B00b-1 pone 0 en las existencias vacías: NOT NULL falla con ORA-02296 si queda algún NULL. B00c y B00d dejan los mismos datos para todos (los estudiantes no tienen la tabla completa ni clientes). La leche con 0 existencias es el caso límite de la sesión; aceite y leche quedan por debajo de la mínima, que es la regla con IF de la S05. Sin la tabla de la S03: B00-R.

BLOQUE 1 · 35 MINUTOS

SELECT INTO y sus errores

Hasta ahora los bloques calculaban con valores escritos en el código. Este bloque los conecta con las tablas: cómo se lleva una fila a variables y qué pasa cuando la consulta no devuelve exactamente una fila.

Transición: en la S03 el SELECT INTO apareció como recurso para %TYPE; hoy se estudia su regla.

BLOQUE 1

Sentencias SQL dentro de un bloque

Pregunta rápida. ¿Se puede escribir un `CREATE TABLE` entre el `BEGIN` y el `END` de un bloque?

El motor PL/SQL ejecuta las variables y el control; cada sentencia SQL del bloque la envía al motor SQL del servidor y recibe el resultado.

SE ESCRIBE DIRECTAMENTE EN EL BLOQUE	NO SE ESCRIBE DIRECTAMENTE
SELECT … INTO · INSERT · UPDATE · DELETE	DDL: CREATE, ALTER, DROP
COMMIT · ROLLBACK · SAVEPOINT	DCL: GRANT, REVOKE

Un `CREATE TABLE` dentro del `BEGIN` produce `PLS-00103: Encountered the symbol "CREATE"`. El DDL se ejecuta fuera del bloque, como en B00.

5 min. Respuesta: no. El DDL dentro de un bloque requiere SQL dinámico, que no forma parte del Corte 1. Las tablas del proyecto se crean con scripts SQL, no con bloques.

SELECT INTO: una fila a variables

```
DECLARE
  v_nombre      productos.nombre%TYPE;
  v_precio      productos.precio%TYPE;
  v_existencias productos.existencias%TYPE;
BEGIN
  SELECT nombre, precio, existencias
    INTO v_nombre, v_precio, v_existencias
    FROM productos
   WHERE codigo = 4;
  DBMS_OUTPUT.PUT_LINE(v_nombre || ' · $' || v_precio
                       || ' · ' || v_existencias || ' unidades');
END;
/
```

```
Aceite 1 L · $15900 · 8 unidades
```

- La consulta debe devolver **exactamente una fila**.
- Las variables reciben los valores **en el orden** de la lista del SELECT.
- Mismo número de columnas que de variables, con tipos compatibles: %TYPE lo garantiza.

Regla de oro. SELECT INTO se usa cuando el WHERE identifica una sola fila: por la llave primaria o con una función de grupo.

6 min. Ejecutar B01. Cambiar el código 4 por 7 y ejecutar de nuevo: Jabón de baño · $5800 · 15.

SELECT INTO con funciones de grupo

Pregunta rápida. Si ningún producto cumple el WHERE, ¿qué devuelven `COUNT(*)` y `SUM(existencias)`?

```
SELECT COUNT(*), SUM(existencias)
  INTO v_cantidad, v_unidades
  FROM productos
 WHERE iva = 0;
```

```
Exentos: 3 productos, 65 unidades
Caso vacío: 0 productos, [] unidades
Con NVL:    0 unidades
```

FUNCIÓN	CON FILAS	SIN FILAS
COUNT(*)	cuántas	0
SUM, AVG, MAX	el valor	NULL

Una función de grupo sin GROUP BY devuelve siempre una fila, aunque ninguna cumpla el WHERE. Por eso no produce error en un `SELECT INTO`.

Regla de oro. Después de un SUM, `NVL(v_total, 0)`: el conjunto vacío da NULL, no cero.

6 min. Ejecutar B02. Señalar los corchetes vacíos: es el NULL de la S03 apareciendo en un dato real. Respuesta: COUNT devuelve 0 y SUM devuelve NULL.

Errores frecuentes del SELECT INTO

ERROR	POR QUÉ OCURRE	CÓMO SE MANIFIESTA	CÓMO SE CORRIGE
Ninguna fila	El WHERE no encuentra filas: el código 9 todavía no existe	`ORA-01403: no data found`	Consultar por un valor que exista, o contar primero con COUNT(*)
Varias filas	El WHERE no identifica una fila: tres productos tienen iva = 0	`ORA-01422: exact fetch returns more than requested number of rows`	Filtrar por la llave primaria, o usar una función de grupo
Falta INTO	Un SELECT dentro del bloque debe dejar el resultado en variables	`PLS-00428: an INTO clause is expected in this SELECT statement`	Agregar INTO con una variable por columna

Los dos primeros son errores de ejecución: el bloque compila y falla al correr. El tercero es de compilación: el bloque no llega a ejecutarse. En la S07 se aprende a atrapar los dos primeros (NO_DATA_FOUND y TOO_MANY_ROWS) en la sección EXCEPTION.

8 min. Ejecutar B03-E1, E2 y E3, y leer cada mensaje completo en voz alta. El Taller 04 pide leer los mismos tres errores con otra tabla.

EJERCICIO · 6 MINUTOS código: COM32_S04_ejemplos.sql · B01 y B02

Ejercicio en clase

PASO	QUÉ HACER	QUÉ DEBE OBSERVAR	QUÉ ERROR PREVIENE
1	En B01, cambie `WHERE codigo = 4` por `WHERE precio > 9000`.	ORA-01422: el café y el aceite cumplen.	Usar SELECT INTO con un filtro que no es único.
2	En B02, cuente los productos con `existencias = 0` y sume sus precios.	1 producto; la suma es 4600.	Suponer que una consulta con WHERE siempre devuelve varias filas.
3	Repita el paso 2 con `existencias < 0` y muestre la suma con y sin NVL.	0 productos; la suma vacía sin NVL, 0 con NVL.	Mostrar un total vacío en lugar de cero.

Síntesis del bloque 1. SELECT INTO lleva una fila a variables y exige exactamente una: ninguna produce ORA-01403 y varias, ORA-01422. Una función de grupo siempre devuelve una fila; su suma vacía es NULL.

9 min con la síntesis. Ruta mínima: solo el paso 1.

BLOQUE 2 · 35 MINUTOS

INSERT, UPDATE y DELETE en el bloque

Ya sabemos leer una fila desde el bloque. Ahora el bloque escribe: inserta, modifica y elimina filas con los valores de sus variables, y decide si los cambios se confirman o se deshacen.

Transición: el SELECT INTO trae datos de la tabla a las variables; el DML lleva los valores de las variables a la tabla.

INSERT con variables

```
DECLARE
  v_codigo      productos.codigo%TYPE      := 9;
  v_nombre      productos.nombre%TYPE      := 'Azúcar 1 kg';
  v_precio      productos.precio%TYPE      := 4800;
  v_existencias productos.existencias%TYPE := 20;
BEGIN
  INSERT INTO productos (codigo, nombre, precio, existencias)
  VALUES (v_codigo, v_nombre, v_precio, v_existencias);
  DBMS_OUTPUT.PUT_LINE('Filas insertadas: ' || SQL%ROWCOUNT);
  COMMIT;
END;
/
```

- Los valores salen de variables declaradas con %TYPE.
- La lista de columnas se escribe siempre: el INSERT no depende del orden de la tabla.
- La columna que no se nombra recibe su DEFAULT: el azúcar queda con iva 0.19.
- Las restricciones de la tabla se aplican igual que fuera del bloque.

```
Filas insertadas: 1
```

4 min. Ejecutar B04 y después B01 con el código 9: ahora sí existe. SQL%ROWCOUNT se explica en el bloque 3; por ahora se lee como «filas afectadas».

Una venta: dos sentencias en un bloque

```
DECLARE
  c_codigo   CONSTANT productos.codigo%TYPE := 4;
  c_cantidad CONSTANT ventas.cantidad%TYPE  := 3;
  v_nombre      productos.nombre%TYPE;
  v_existencias productos.existencias%TYPE;
BEGIN
  INSERT INTO ventas (numero, codigo, cantidad, fecha)
  VALUES (1, c_codigo, c_cantidad, DATE '2026-10-01');

  UPDATE productos
     SET existencias = existencias - c_cantidad
   WHERE codigo = c_codigo
  RETURNING nombre, existencias INTO v_nombre, v_existencias;
  ...
  COMMIT;
END;
/
```

Una operación del negocio suele ser varias sentencias: registrar la venta y descontar las existencias. Las dos van en el mismo bloque y se confirman juntas.

```
Venta registrada: 3 × Aceite 1 L
Existencias restantes: 5
```

5 min. Ejecutar B05. Mostrar la tabla ventas con una fila y el aceite con 5 existencias.

RETURNING INTO

RETURNING … INTO devuelve a variables los valores de la fila recién insertada, modificada o eliminada, en la misma sentencia.

```
-- sin RETURNING: dos sentencias
UPDATE productos SET existencias = existencias - 3
 WHERE codigo = 4;
SELECT existencias INTO v_existencias
  FROM productos WHERE codigo = 4;
```

```
-- con RETURNING: una sentencia
UPDATE productos SET existencias = existencias - 3
 WHERE codigo = 4
RETURNING existencias INTO v_existencias;
```

CONDICIÓN	DETALLE
Filas	Con INTO de variables escalares, la sentencia debe afectar una fila
Valor devuelto	En UPDATE, el valor **después** del cambio: 5, no 8
Uso típico	Leer el saldo, el stock o el estado que quedó, para informarlo o validarlo

4 min. El Taller 04 (punto 3) pide un UPDATE con RETURNING INTO sobre otra tabla.

DELETE, COMMIT y ROLLBACK

> **Pregunta rápida.** Si el bloque elimina una fila y después ejecuta ROLLBACK, ¿cuántos productos quedan en la tabla?

```
DELETE FROM productos WHERE existencias = 0;
-- Filas eliminadas: 1
SELECT COUNT(*) INTO v_total FROM productos;
-- Productos antes del ROLLBACK: 7
ROLLBACK;
SELECT COUNT(*) INTO v_total FROM productos;
-- Productos después del ROLLBACK: 8
```

Una **transacción** es la unidad de trabajo que agrupa las sentencias DML hasta que se confirma o se deshace.

SENTENCIA	EFECTO
`COMMIT`	Hace permanentes los cambios y los deja visibles para otras sesiones
`ROLLBACK`	Deshace todos los cambios no confirmados
`SAVEPOINT s1`	Marca un punto; `ROLLBACK TO s1` deshace solo desde ahí

Analogía: el cambio no confirmado es como el borrador de un correo: solo lo ve quien lo escribe, y puede enviarse (COMMIT) o descartarse (ROLLBACK).

> **6 min.** Ejecutar B06. Respuesta: 8, porque el ROLLBACK deshace el DELETE. La comprobación va en el mismo bloque para que no dependa de cómo el entorno maneja la sesión entre ejecuciones.

Si una sentencia falla, el bloque completo se revierte

```
BEGIN
  INSERT INTO ventas (numero, codigo, cantidad, fecha)
  VALUES (2, 6, 1, DATE '2026-10-01');
  UPDATE productos SET existencias = existencias - 1
   WHERE codigo = 6;                    -- la leche tiene 0
  COMMIT;
END;
/
```

```
ORA-02290: check constraint (…CK_PRODUCTOS_EXISTENCIAS) violated
```

- El INSERT de la venta 2 se ejecutó bien.
- El UPDATE dejaría −1 existencias: la restricción CHECK lo rechaza.
- El error no se atrapa, así que Oracle deshace todo lo que hizo el bloque.
- `SELECT COUNT(*) FROM ventas` sigue en 1.

Regla de oro. La restricción de la tabla es la primera validación del negocio: el bloque no puede saltársela.

5 min. Ejecutar B05-E y después el SELECT COUNT(*) de ventas. Esto es lo que se aprovecha en el proyecto: la regla «no vender sin existencias» la garantiza un CHECK aunque el bloque se equivoque.

Errores frecuentes con DML en el bloque

ERROR	POR QUÉ OCURRE	CÓMO SE MANIFIESTA	CÓMO SE CORRIGE
Llave repetida	Se ejecuta dos veces el mismo INSERT con la misma PK	`ORA-00001: unique constraint (…) violated`	Usar otro código, o consultar antes si existe
Restricción CHECK	El valor nuevo no cumple la condición de la columna	`ORA-02290: check constraint (…) violated`	Corregir el dato; la restricción se mantiene
FK inexistente	La venta referencia un producto que no existe	`ORA-02291: integrity constraint (…) violated - parent key not found`	Insertar primero la fila padre
Falta COMMIT	Los cambios quedan sin confirmar y pueden perderse al cerrar la sesión	Sin mensaje: otra sesión no ve los datos	Terminar la operación con COMMIT

4 min. Ejecutar B04 por segunda vez para mostrar ORA-00001 en vivo.

EJERCICIO · 6 MINUTOS código: COM32_S04_ejemplos.sql · B04 a B06

Ejercicio en clase

PASO	QUÉ HACER	QUÉ DEBE OBSERVAR	QUÉ ERROR PREVIENE
1	Registre la venta 3: 2 unidades del producto 7, con INSERT en ventas y UPDATE con RETURNING.	Existencias restantes: 13.	Descontar el stock sin registrar la venta, o al revés.
2	Cambie el producto por el 99 y ejecute de nuevo con la venta 4.	ORA-02291: el producto 99 no existe.	Registrar ventas de productos inexistentes.
3	Elimine en un bloque los productos con precio menor que 3000, muestre cuántos eliminó y termine con ROLLBACK.	Filas eliminadas: 1 (arroz); la tabla vuelve a 8 productos.	Borrar datos de prueba sin poder recuperarlos.

Síntesis del bloque 2. El bloque escribe en las tablas con los valores de sus variables; las restricciones se aplican igual. Los cambios se confirman con COMMIT o se deshacen con ROLLBACK, y un error no atrapado revierte el bloque completo.

6 min con la síntesis. Ruta mínima: solo el paso 1.

BLOQUE 3 · 25 MINUTOS

Cursor implícito y sus atributos

El bloque ya consulta y modifica. Falta saber qué pasó: cuántas filas afectó cada sentencia y si encontró alguna. Ese dato lo guarda el cursor implícito.

Transición: en B04 apareció SQL%ROWCOUNT sin explicación; aquí se define.

BLOQUE 3 · CONCEPTO

Qué es un cursor implícito

> **Pregunta rápida.** Cuando un bloque ejecuta un UPDATE, ¿dónde queda registrado cuántas filas modificó?

Un **cursor** es el área de trabajo que Oracle abre para procesar una sentencia SQL y guardar su resultado y su estado. Hay dos clases:

	CURSOR IMPLÍCITO	CURSOR EXPLÍCITO
Quién lo abre	Oracle, en cada SELECT INTO, INSERT, UPDATE y DELETE	El programador, con DECLARE, OPEN, FETCH y CLOSE
Nombre	Siempre SQL	El que se declara: `cur_ventas`
Filas	Una fila (SELECT INTO) o las que afecte el DML	Recorre varias filas, una a una
Sesión	Hoy, S04	S06

Respuesta: en el cursor implícito SQL, que describe siempre la última sentencia SQL ejecutada por el bloque.

4 min. Dejar claro que el explícito es de la S06: hoy no se declara ningún cursor.

SQL%ROWCOUNT, SQL%FOUND y SQL%NOTFOUND

ATRIBUTO	TIPO	VALOR DESPUÉS DE LA SENTENCIA
SQL%ROWCOUNT	Entero	Número de filas afectadas o devueltas
SQL%FOUND	BOOLEAN	TRUE si afectó al menos una fila
SQL%NOTFOUND	BOOLEAN	TRUE si no afectó ninguna fila
SQL%ISOPEN	BOOLEAN	Siempre FALSE: Oracle cierra el cursor implícito al terminar la sentencia

```
UPDATE productos SET precio = ROUND(precio * 1.05)
 WHERE iva = 0.05;
v_filas := SQL%ROWCOUNT;
DBMS_OUTPUT.PUT_LINE('filas ' || v_filas || ' · FOUND ' ||
  CASE WHEN SQL%FOUND THEN 'sí' ELSE 'no' END);
```

```
Tarifa 5 %:  filas 2 · FOUND sí
```

Los BOOLEAN se muestran con CASE, como en la S03. En la S05 se usan en un IF para decidir qué hacer.

4 min. Ejecutar la primera mitad de B07: café y atún suben 5 %.

Un UPDATE sin filas no es un error

Pregunta rápida. Un UPDATE cuyo WHERE no encuentra ninguna fila, ¿produce un error como el ORA-01403 del SELECT INTO?

```
UPDATE productos SET precio = ROUND(precio * 1.05)
 WHERE iva = 0.19 AND existencias > 100;
DBMS_OUTPUT.PUT_LINE('filas ' || SQL%ROWCOUNT ||
  ' · FOUND '    || CASE WHEN SQL%FOUND    THEN 'sí' ELSE 'no' END ||
  ' · NOTFOUND ' || CASE WHEN SQL%NOTFOUND THEN 'sí' ELSE 'no' END);
```

```
Sin filas:   filas 0 · FOUND no · NOTFOUND sí
```

SENTENCIA	SIN FILAS
SELECT INTO	Error ORA-01403
UPDATE · DELETE	Termina bien; SQL%ROWCOUNT = 0

Regla de oro. Después de un UPDATE o DELETE que debe afectar filas, se lee SQL%ROWCOUNT: el bloque no avisa por sí solo que no encontró nada.

4 min. Respuesta: no. Es la causa del error silencioso más común: «el bloque corrió, pero no cambió nada».

SQL%ROWCOUNT describe la última sentencia

```
UPDATE productos SET existencias = existencias + 5
 WHERE iva = 0;
v_filas := SQL%ROWCOUNT;
COMMIT;
DBMS_OUTPUT.PUT_LINE('Guardado antes del COMMIT: ' || v_filas);
DBMS_OUTPUT.PUT_LINE('Leído después del COMMIT:  ' || SQL%ROWCOUNT);
```

```
Guardado antes del COMMIT: 3
Leído después del COMMIT:  0
```

Cada sentencia SQL reemplaza los atributos de la anterior. COMMIT también es una sentencia SQL: después de ella, SQL%ROWCOUNT ya no describe el UPDATE.

> El valor se guarda en una variable inmediatamente después de la sentencia que interesa.

2 min. RUTA MÍNIMA: diapositiva para lectura si el tiempo no alcanza.

Variable con el nombre de una columna

> **Pregunta rápida.** Este bloque quiere sumar 10 existencias al producto 4. ¿Cuántas filas modifica?

```
DECLARE
  codigo productos.codigo%TYPE := 4;
BEGIN
  UPDATE productos
     SET existencias = existencias + 10
   WHERE codigo = codigo;
  DBMS_OUTPUT.PUT_LINE('Filas modificadas: '
                       || SQL%ROWCOUNT);
  ROLLBACK;
END;
```

```
Filas modificadas: 8
```

Dentro de una sentencia SQL, Oracle resuelve un nombre primero como **columna**. `codigo = codigo` compara la columna consigo misma y es verdadero en todas las filas.

No aparece ningún mensaje de error: solo SQL%ROWCOUNT lo delata.

> **Regla de oro.** Ninguna variable se llama igual que una columna: `v_codigo`, nunca `codigo`.

4 min. Pedir la predicción antes de ejecutar B08-E. Respuesta: 8. Ejecutar después la versión con v_codigo: 1 fila. Es el fundamento de la convención v_ que se evalúa desde la S02.

EJERCICIO · 4 MINUTOS código: COM32_S04_ejemplos.sql · B07 y B08

Ejercicio en clase

PASO	QUÉ HACER	QUÉ DEBE OBSERVAR	QUÉ ERROR PREVIENE
1	Suba 10 % el precio de los productos con existencias mayores que 20, muestre SQL%ROWCOUNT y termine con ROLLBACK.	Filas: 3 (arroz, panela y atún).	Suponer el número de filas sin leerlo.
2	Repita con existencias mayores que 500 y muestre SQL%FOUND con CASE.	Filas: 0 · FOUND no.	Tomar «el bloque terminó bien» como «el bloque cambió algo».
3	En B08-E, reemplace `codigo` por `v_codigo` solo en la declaración.	Siguen modificándose 8 filas: el WHERE todavía compara la columna consigo misma. Corrija también el WHERE: 1 fila.	Corregir el nombre a medias.

Síntesis del bloque 3. El cursor implícito SQL describe la última sentencia: SQL%ROWCOUNT dice cuántas filas afectó, y SQL%FOUND y SQL%NOTFOUND si afectó alguna. Un DML sin filas no es error, y una variable con nombre de columna cambia el sentido del WHERE.

4 min con la síntesis. Existencias tras B07b: arroz 45, panela 30, atún 30, azúcar 20, leche 5. Mayores que 20: arroz, panela y atún.

ANTES DEL PROYECTO Y DEL TALLER

Reglas de oro

1. `SELECT INTO` exige exactamente una fila: se filtra por la llave o se usa una función de grupo.
2. Una suma sobre un conjunto vacío es NULL: `NVL(v_total, 0)`.
3. Una operación del negocio va en un bloque y termina con `COMMIT`; un error no atrapado la revierte completa.
4. Un UPDATE o DELETE sin filas no es error: se lee `SQL%ROWCOUNT` justo después de la sentencia.
5. Ninguna variable se llama igual que una columna.

2 min. Las cinco reglas se evalúan en el Taller 04 y en la Entrega 1.

PROYECTO INTEGRADOR · 30 MINUTOS

Estructura completa de la entrega

```
COM32_EntregaN_GrupoN.zip
├─ 00_limpiar.sql ........... E1
├─ 01_esquema.sql ........... E1
├─ 02_datos.sql ............. E1
├─ 03_validaciones.sql ...... E1
├─ 04_procedimientos.sql .... E2
├─ 05_funciones.sql ......... E2
├─ 06_cursores.sql .......... E2
├─ 07_permisos.sql .......... E2  (Supabase)
├─ 08_paquete.sql ........... F
├─ 09_disparadores.sql ...... F
├─ 10_pruebas.sql ........... F
├─ informe_entregaN.pdf
└─ manual_tecnico.pdf ....... F
```

ENTREGA	CIERRE	PESO
E1	Mié 14 oct 2026, 23:59	15 % del Corte 1
E2	Jue 12 nov 2026, 23:59	Corte 2
Final	Mié 9 dic 2026, 23:59 · sustentación jue 10 dic	Corte 3

> En cada entrega se suben **todos** los scripts existentes. Se califica ejecutando `00_limpiar.sql` y luego los demás, en orden, en un esquema limpio.

Documentos en el aula: `COM32_ProyectoIntegrador_lineamientos.pdf` y `COM32_proyecto_plantilla.zip`.

8 min. La Entrega 1 se movió del 8 al 14 de octubre: IF, CASE y los ciclos se estudian en la S05. La plantilla trae el encabezado de cada script.

PROYECTO INTEGRADOR · ENTREGA 1

Entrega 1: esquema, datos y validaciones

REQ.	CRITERIO DE ACEPTACIÓN	PESO
R1	00, 01 y 02 corren en limpio; 4 tablas o más con PK y FK con nombre, una 1:N, una N:M con tabla intermedia, una entidad débil y 15 filas por tabla	0.6
R2	03 con cinco bloques o más, uno por regla, con el resultado declarado. Entre todos: SELECT INTO, función de grupo, DML con SQL%ROWCOUNT, IF o CASE y un ciclo	1.4
R3	Nomenclatura, %TYPE, una constante, encabezado y comentario de regla en cada bloque	1.0
R4	NOT NULL, UNIQUE, CHECK y DEFAULT del negocio; datos que recorren cada rama	0.75
R5	Informe completo, exportado a PDF	0.75
R6	Puntualidad, nombre y contenido del archivo	0.5
	Total	**5.0**

Grupal · Cierre **miércoles 14 de octubre de 2026, 23:59** · Archivo `COM32_Entrega1_GrupoN.zip` · En la S05 cada grupo muestra 01 y 02 ejecutados en APEX.

7 min. Leer la condición de cada regla: encima del bloque van la regla, las tablas y el resultado esperado. Descuento de 0.3 por variable con nombre de columna o por SYSDATE en un resultado esperado.

Hoy: crear y poblar el esquema en APEX

1. Descargue `COM32_proyecto_plantilla.zip` y complete el encabezado de `01_esquema.sql`.
2. Escriba las tablas en orden de dependencia: primero las que no tienen FK.
3. En APEX: **SQL Workshop → SQL Scripts → Create**, pegue el script y ejecútelo. Revise el resumen: cero errores.
4. Repita con `02_datos.sql`: por ahora, tres filas por tabla para probar las FK.
5. Escriba en **SQL Commands** la primera regla con la plantilla de B09.

Sin espacio APEX aprobado

El grupo trabaja en FreeSQL con el mismo script; el esquema se pasa a APEX cuando llegue la aprobación.

En APEX

No se escribe `SET SERVEROUTPUT ON`: la salida aparece en «Results». SQL Commands confirma cada sentencia por defecto (casilla Autocommit).

10 min. Recorrer los grupos: revisar que cada tabla tiene PK con nombre y que el orden de creación respeta las FK. Quien no tiene APEX aprobado trabaja en FreeSQL.

ANTES DEL TALLER · 5 MINUTOS código: COM32_S03_ejemplos.sql

Lo que el Taller 04 toma de la S03

El Taller 03 no se publicó. El Taller 04 integra las dos sesiones: estas son las técnicas de la S03 que se evalúan, y dónde.

TÉCNICA DE LA S03	RECORDATORIO	BLOQUE S03	PUNTO DEL TALLER
Variables ancladas	`v_titulo libros.titulo%TYPE` hereda tipo y tamaño	B05	P1 · P3 · P5a
Número a texto	`TO_CHAR(v_precio, 'FM$999G999', 'NLS_NUMERIC_CHARACTERS='',.''')`	B10	P1
Fecha a texto	`TO_CHAR(v_fecha, 'fmday, dd "de" month "de" yyyy', 'NLS_DATE_LANGUAGE=SPANISH')`	B10	P1
Texto a número y a fecha	`TO_NUMBER('89.900', '99G999', …)` · `TO_DATE('01/10/2026', 'DD/MM/YYYY')`	B11	P3
Precedencia y NULL	* antes que || · NULL + 1 es NULL · NVL	B06 a B08	P2
Errores de tipo y conversión	`ORA-06502` (no cabe) · `ORA-01839` (fecha inexistente)	B05-E · B12	P5

> Las conversiones se escriben explícitas, con máscara y parámetro regional: el resultado no puede depender de la configuración del servidor.

5 min. Proyectar a las 20:20. El script de la S03 sigue en el aula como consulta. Esta diapositiva no se recorta: sin ella, los puntos 1 y 3 del taller usan técnicas de una sesión que no se evaluó.

EVALUACIÓN DE LA SESIÓN

Taller 04

Variables, SQL en PL/SQL y cursor implícito · integrador S03 y S04

RA1 · CE1 · CE2 · CE3 | INDIVIDUAL | ESCALA 1.0 – 5.0 | FREESQL (FREESQL.COM)

Se trabaja sobre el Word del aula, `COM32_S04_taller04.docx`, sobre la tabla `libros` que crea `COM32_S04_taller04_datos.sql`.

Los puntos se resuelven en orden: los puntos 3 y 4 modifican la tabla, y los resultados esperados de los puntos 4 y 5 parten del estado que deja el anterior.

Presentar en 5 minutos, 20:25 a 20:30. El taller reemplaza al Taller 03, que no se publicó.

TALLER 04

Puntos y pesos

PUNTO	QUÉ SE PIDE	SESIÓN	CRITERIO	PESO
1	**%TYPE, SELECT INTO y TO_CHAR:** un libro con precio y fecha convertidos	S03 · S04	CE1 · CE3	1.0
2	**Operadores, NULL y funciones de grupo:** predecir seis resultados y explicar	S03 · S04	CE2	0.8
3	**INSERT con TO_NUMBER y TO_DATE; UPDATE con RETURNING INTO**	S03 · S04	CE3	1.0
4	**Cursor implícito y conflicto de nombres:** SQL%ROWCOUNT, FOUND y NOTFOUND; un bloque con error lógico	S04	CE2 · CE3	1.2
5	**Lectura de errores:** ORA-06502, ORA-01839, ORA-01403, ORA-01422 y ORA-02290	S03 · S04	CE1	1.0
	Total			**5.0**

Cada punto remite a un bloque del script de la S03 o de la S04, con otra tabla y otros datos.

3 min. Si un punto no funciona, revisar el bloque del script indicado en el enunciado antes de preguntar.

TALLER 04

Condiciones y entrega

1. Descargue `COM32_S04_taller04.docx` y `COM32_S04_taller04_datos.sql` de la actividad «Taller 04».
2. Ejecute el script de datos una vez, completo, en FreeSQL.
3. Resuelva los puntos en orden y copie el código dentro del Word, **como texto**.
4. Pegue la captura solo en el punto 4 (a).
5. Suba el mismo Word como `COM32_Taller04_Apellido.docx`.

Cierre

Miércoles 7 de octubre de 2026, 23:59

Consultas

Foro de la actividad, con el mensaje de error completo y el bloque que lo produce.

Si necesita empezar de nuevo: `DROP TABLE libros PURGE;` y vuelva a ejecutar el script de datos.

2 min. Durante el taller: a las 21:00 y a las 21:30, verificar que cada estudiante tiene los puntos 1 y 3 ejecutados: son los que integran las conversiones de la S03. A las 21:45, avisar que quedan diez minutos.

CIERRE · 21:55

Síntesis de la sesión

- `SELECT INTO` exige una fila; las funciones de grupo siempre devuelven una.
- El bloque inserta, modifica y elimina con sus variables; COMMIT confirma y ROLLBACK deshace.
- El cursor implícito SQL dice cuántas filas afectó la última sentencia.

Próxima sesión · S05 · jueves 8 de octubre

Estructuras de control: `IF`, `CASE`, `LOOP`, `WHILE` y `FOR`. Cada grupo muestra `01_esquema.sql` y `02_datos.sql` ejecutados en APEX. Taller 04: miércoles 7 de octubre, 23:59. Entrega 1: miércoles 14 de octubre, 23:59.

5 min. Pregunta de salida: por qué un UPDATE puede terminar sin error y no cambiar nada. Respuesta: su WHERE no encontró filas; SQL%ROWCOUNT es 0.